Exploring Phase-Specific Configuration of Interactive Evolutionary Multiobjective Optimization Methods

Giomara Lárraga
giomara.g.larraga-maldonado@jyu.fi
University of Jyvaskyla
Faculty of Information Technology
Finland

Kaisa Miettinen
kaisa.miettinen@jyu.fi
University of Jyvaskyla
Faculty of Information Technology
Finland

Abstract

Interactive evolutionary multiobjective methods enable a decision maker to solve optimization problems involving multiple conflicting objective functions by iteratively incorporating preference information. When applying interactive methods, two phases can often be identified: a learning phase, where a decision maker gains insights on trade-offs and identifies a region of interest based on their preferences, and a decision phase focused on fine-tuning and selecting the most preferred solution. The configuration of evolutionary operators, like selection, crossover, and mutation, heavily influences the performance of evolutionary methods. However, despite extensive research on parameter tuning, identifying optimal configurations for these operators within interactive methods while accounting for the specific goals of each phase has not been studied. This study introduces a framework for the automatic configuration of interactive methods, taking the first step toward addressing this research gap. The framework systematically identifies phase-specific optimal configurations by combining the *PHI* indicator with the irace automatic configuration tool. Experiments with interactive RVEA and interactive RNSGA-II on problems involving three, five, and seven objective functions reveal notable differences in optimal configurations between the learning and decision phases. These findings lay a foundation for enhancing the performance of interactive evolutionary multiobjective methods and highlight the importance of phase-specific configurations.

CCS Concepts

• **Mathematics of computing** → **Bio-inspired optimization**; • **Applied computing** → **Multi-criterion optimization and decision-making**.

Keywords

multiobjective optimization, decision making, interactive methods, evolutionary algorithms

ACM Reference Format:
Giomara Lárraga and Kaisa Miettinen. 2025. Exploring Phase-Specific Configuration of Interactive Evolutionary Multiobjective Optimization Methods . In *Genetic and Evolutionary Computation Conference (GECCO '25), July 14–18, 2025, Malaga, Spain.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3712256.3726367



This work is licensed under a Creative Commons Attribution 4.0 International License.
GECCO '25, July 14–18, 2025, Malaga, Spain
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1465-8/2025/07
https://doi.org/10.1145/3712256.3726367

1 Introduction

Evolutionary multiobjective optimization methods have become popular for addressing problems involving multiple conflicting objective functions that must be optimized simultaneously [16, 18]. Typically, the conflicts make it impossible to identify a single solution that optimizes all objective functions simultaneously. Instead, a set of trade-off solutions, known as Pareto optimal solutions, exists where improving one objective function is possible by only impairing at least some other function. The image of this set of solutions in the objective space is called a Pareto front.

Solving multiobjective optimization problems aims to support a domain expert, referred to as a decision maker, in identifying the most preferred solution from the Pareto optimal set that best aligns with their preferences. According to the role of a decision maker during the solution process, multiobjective optimization methods can be categorized into [37]: no-preference, a priori, a posteriori and interactive ones. No-preference methods do not consider the preferences of a decision maker, while a priori and a posteriori methods apply preference information before and after the solution process, respectively. Interactive methods allow a decision maker to provide their preferences iteratively during the solution process. In each iteration, a decision maker provides preference information and gets a set of solutions reflecting them. These solutions help a decision maker learn about feasible trade-offs, refine their preferences, and guide the process toward the most preferred solution. Often, two phases can be identified during an interactive solution process [38]: a learning phase, where a decision maker explores the Pareto optimal set to find a region of interest, and a decision phase, where preferences are fine-tuned to identify the most preferred solution in the region of interest.

There are multiple ways in which a decision maker can express their preferences. According to [8], reference points stand out as the most widely used way. A reference point comprises desired values, also referred to as aspiration levels, for each objective function [44]. Other methods for expressing preferences include defining desirable ranges for each objective function (e.g.,[25, 33]) and selecting preferred or non-preferred solutions (e.g., [24, 25]).

The performance of evolutionary methods heavily relies on properly configuring their control parameters [23]. To facilitate the selection of the best configuration, various approaches have been developed for automatic method configuration, including experimental design [1, 17], racing algorithms [12], and other strategies [6, 28, 43]. However, these approaches have not been applied to interactive methods.

Interactive methods typically use a fixed configuration throughout the entire solution process. Since many interactive methods

are adaptations of existing a posteriori evolutionary multiobjective methods [8, 30], their configurations often inherit those of the base method. However, it has not been explored whether the different phases of the interactive solution process (reflecting different needs of a decision maker) require distinct configurations.

The automatic configuration of interactive methods poses unique challenges, as their performance depends on how effectively they adapt to the changing preferences of a decision maker [5]. Evaluating the performance of these methods remains an open research area, with few indicators that consider essential characteristics such as aligning solutions with preferences, exploring various regions of the Pareto optimal set, and fine-tuning solutions. A notable exception is the preference-based hypervolume indicator (*PHI*) [4], which can be utilized to assess interactive evolutionary methods considering the different goals of the learning and decision phases.

In this study, we investigate how the characteristics of interactive methods influence the selection of configurations. We show that applying configurations from a posteriori and a priori methods to interactive methods is insufficient, especially when considering the evolving preference information provided by a decision maker during the solution process. Specifically, we explore how different phases of the interactive process require distinct configurations to effectively accommodate the changing preferences of a decision maker. We propose a framework for the automatic configuration of interactive methods using reference points as preference information. This framework utilizes the irace tool [36] to identify optimal configurations for the learning and decision phases separately. *PHI* values are utilized by irace to assess the performance of an interactive method under different configurations. To achieve an accurate representation of both phases, we use an artificial decision maker (ADM) [2] to generate different sets of reference points based on the phase in question. During the learning phase, the ADM places reference points in unexplored regions of the Pareto optimal set. In the decision phase, it focuses on generating reference points within a region of interest. Although it can be challenging to identify the exact transition between the learning and decision phases in practice, this can be effectively achieved by observing the preferences provided by a decision maker throughout the process.

We apply the proposed framework to find optimal configurations for two interactive methods: interactive RVEA [25] and interactive RNSGA-II [20], using WFG [27] and DTLZ [21] benchmark problems with three, five and seven objective functions. We analyze the results to identify differences in the optimal configurations of selection, mutation, and crossover operators between the learning and decision phases. Additionally, we compare the performance of phase-specific configurations with that of a single, general configuration optimized for the entire solution process.

The key contributions of this study are as follows:

- A novel framework to automatically identify optimal configurations for the learning and decision phases of interactive methods.
- A detailed analysis of the differences in optimal configurations for evolutionary operators, such as selection, crossover, and mutation, between the learning and decision phases.
- A comprehensive comparison of phase-specific configurations against general configurations, identifying their strengths and weaknesses.

The remainder of this paper is organized as follows: Section 2 is devoted to related work, including the automatic configuration of evolutionary multiobjective optimization methods, the interactive methods utilized in this study, the *PHI* indicator, and ADMs. We provide a comprehensive overview of the proposed framework in Section 3. Section 4 details the experimental setup and presents the obtained results, which are then discussed in Section 5. Finally, we conclude in Section 6 with key insights and future research directions.

2 Background

This section reviews key concepts relevant to the proposed framework. We first examine prior work on the automatic configuration of evolutionary multiobjective optimization methods. Next, we introduce the two interactive methods utilized in our study. Finally, we describe the role of the *PHI* indicator and ADMs in assessing the performance of interactive methods.

2.1 Automatic configuration of evolutionary multiobjective methods

Evolutionary methods have many parameters. The automatic configuration of evolutionary methods aims to simplify the process of fine-tuning their parameters. Implementing evolutionary methods using a component-wise design [11], enables both parameter optimization and the design of new method variants. This approach involves using a set of configurable components that can take values within defined ranges. For instance, the basic NSGA-II method [19] employs tournament selection, simulated binary crossover (SBX) and polynomial mutation. By implementing these evolutionary operators as configurable components, one can explore variants of NSGA-II that could potentially enhance its performance.

In multiobjective optimization, this approach has been applied to methods such as MOACO [9, 34, 35], MOPSO [22, 42], NSGA-II [41], and MOEA/D [10, 31], often using the irace tool [36] for automatic configuration and design. irace employs a statistical method called iterated racing [13] to explore the parameter space and identify the best-performing configurations efficiently. To achieve this, irace requires a measurable performance indicator to minimize, a list of parameters to configure, a stopping criterion for the method, and a set of test problems.

2.2 Interactive methods

The selection of methods to utilize in this study was constrained by the lack of availability of open-source implementations of interactive methods. DESDEO [39], a software framework for interactive multiobjective optimization, offers implementations of multiple interactive methods and features a modular structure that facilitates the implementation of additional methods. This made it straightforward to adopt methods already available within DESDEO, such as interactive RVEA (iRVEA) [25]. However, since DESDEO included only decomposition-based methods, we extended its capabilities by implementing an interactive version of RNSGA-II [20] to incorporate a dominance-based method into the experiments.

iRVEA is an interactive method based on the reference vector guided evolutionary algorithm (RVEA) [15]. In the first iteration, RVEA is utilized to get an approximation of the Pareto optimal set by utilizing a set of uniformly distributed unit reference vectors.

A subset of the obtained solutions is then presented to a decision maker, who can either stop the process by selecting their most preferred solution or provide preference information. Preferences can be expressed in the form of reference points, preferred ranges, or preferred/non-preferred solutions. The decision maker provides preference in one of the four ways. iRVEA starts another iteration by updating the reference vectors according to the preference information and evolving the population from the previous iteration for a certain number of generations. This iterative process is repeated until a decision maker selects the most preferred solution. While iRVEA does not specify default evolutionary operators, RVEA uses random selection, SBX and polynomial mutation.

R-NSGA-II [20] is an a priori variant of NSGA-II designed to focus the search on a region of interest specified by a decision maker using reference points. The key modification is an updated crowding distance operator that prioritizes solutions near the reference point. Similar to NSGA-II, R-NSGA-II uses tournament selection, SBX crossover and polynomial mutation for real-valued problems.

In this study, we utilize an interactive version of R-NSGA-II, referred to as iRNSGA-II, which iteratively adjusts to new reference points provided by a decision maker. Each iteration of iRNSGA-II consists of utilizing the a priori R-NSGA-II for a certain number of generations to generate a set of solutions near the reference point provided by a decision maker. At the end of each iteration, the final population becomes the initial population for the next iteration, allowing the method to have a similar behavior to an interactive method.

While iRNSGA-II can be useful for experimentation involving benchmark problems and ADMs, it is acknowledged that its practicality in real-world applications may be limited [30]. Additional measures, such as reducing the cognitive load on decision makers and considering different types of preference information, are necessary to enhance its applicability in practice [3, 29].

2.3 Performance evaluation of interactive evolutionary methods

Assessing the performance of interactive methods introduces unique challenges compared to a priori or a posteriori methods [3]. Traditional performance indicators, like hypervolume [47] and the inverted generational distance [14], which measure convergence and diversity across the entire Pareto optimal set, are inadequate for interactive methods as they do not consider how closely solutions align with the preferences of a decision maker provided at each iteration. While some indicators have been proposed for a priori evolutionary methods [7, 26, 32, 40, 46], they typically assess the quality of a region of interest and fail to address the distinct requirements of the learning and decision phases of interactive methods [5].

The *PHI* indicator [4] assesses the performance of interactive evolutionary methods, where preferences are expressed as reference points. *PHI* focuses on a specific area defined by the reference point and a "dystopian" point (a hypothetical point in the objective space with components worse than in any Pareto optimal solution, in practice, any of the solutions found by the optimization method). It assesses solutions by categorizing them as those dominating the reference point, those being dominated by it, or neither. The indicator calculates the hypervolume, dividing it into positive and negative components to reflect the quality of solutions in relation to the reference point. The final *PHI* value indicates how well the method achieved the preferences of a decision maker by considering both how well the desired area is covered and whether any solutions dominate the preferred point. Since *PHI* is based on hypervolume, a higher value indicates better performance. For more details on calculating the *PHI* value, see [4].

Additionally, *PHI* can assess different phases of an interactive solution process: the learning phase using the PHI_{RS} indicator, which tracks changes in *PHI* values across generations, and the decision phase using the PHI_{FD} indicator, which measures the similarity between reference points and the final solution.

ADMs complement the performance evaluation of interactive methods by simulating the behavior of decision makers in a controlled and repeatable manner. ADMs dynamically generate preferences to enable systematic testing of interactive methods, avoiding potential biases and fatigue that can affect real decision makers. In this study, we use the ADM proposed in [2], which employs distinct strategies for learning and decision phases. During the learning phase, the ADM iteratively constructs a composite front of all Pareto optimal solutions from the methods being compared, identifying unexplored regions to generate new reference points. In the decision phase, the ADM shifts focus to refining solutions within the most explored region of interest, generating reference points that guide the methods toward the most preferred solution. Throughout both phases, the ADM assesses the performance of the compared methods using indicators such as the R-metric [32] or *PHI*. At the end of the solution process, cumulative indicators are calculated to compare the overall effectiveness of the methods across both the learning and decision phases.

3 Proposed framework

In this section, we present a novel framework for the automatic phase-specific configuration of interactive methods. The framework is designed to identify optimal configurations for the learning and decision phases separately and can be utilized for different interactive methods. The structure of the framework is illustrated in Figure 1, and its steps are detailed below.

1. Define test instances: The first step involves defining a set of test instances for evaluating phase-specific configurations. Each test instance includes:

- The problem description, including the number of variables and objective functions.
- A stopping criterion, such as the number of objective function evaluations or generations.
- The phase in question (learning or decision phase).
- A set of reference points tailored to the specific phase.

Reference points are generated in the next step. For each problem, separate test instances are created for learning and decision phases.

2. Generate reference points: Phase-specific reference points are essential for directing the optimization process and accurately reflecting the goals of the learning and decision phases. These points are generated using the ADM proposed in [2], which analyzes sets of solutions produced by multiple interactive methods to compute reference points tailored to each phase. As a result, two distinct

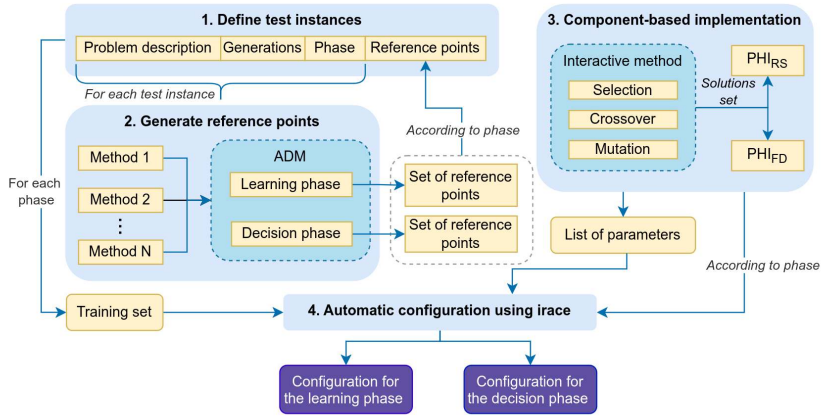


Figure 1: Proposed framework for the automatic configuration of interactive methods.

sets of reference points are obtained: one tailored for the learning phase and another for the decision phase.

3. Component-wise implementation: Interactive methods are implemented using a component-wise design, allowing evolutionary operators (e.g., selection, crossover, mutation) to be treated as interchangeable and tunable components. A list of configurable parameters should be defined, specifying all components and their possible values. The performance of each configuration is evaluated using PHI_{RS} in the learning phase and PHI_{FD} in the decision phase. These indicators measure how effectively solutions align with the phase-specific reference points. To ensure compatibility with irace, which requires minimizing the value of a performance indicator, both PHI_{RS} and PHI_{FD} are multiplied by -1.

4. Automatic configuration using irace: The irace tool utilizes the list of parameters and the component-wise implementation described in the previous step to find the optimal configurations. Test instances from the training set are employed to determine these configurations, while an optional validation set can be used to evaluate the resulting configurations. By systematically exploring the parameter space, irace identifies phase-specific configurations that enhance the performance of an interactive method.

4 Experimentation

We considered a total of 16 test problems for the experimentation, which were selected from the DTLZ [21] and WFG [27] suites, covering diverse Pareto front geometries and complexities. The problems were divided into two groups in a largely arbitrary manner, ensuring a mix of different characteristics in both sets. The training set consisted of DTLZ1, DTLZ2, DTLZ4, DTLZ5, WFG1, WFG2, WFG4, and WFG6, while the validation set included DTLZ3, DTLZ6, DTLZ7, WFG3, WFG5, WFG7, WFG8, and WFG9.

Each test problem was instantiated twice, once for the learning phase and once for the decision phase. Three experimental setups varying the number of objective functions (three, five, and seven) were utilized. The number of variables for both DTLZ and WFG problems was defined as $M+k-1$, where M is a predefined parameter and k is the number of objective functions. For DTLZ1, $M = 5$, while for all other DTLZ problems, $M = 10$. Regarding WFG problems, $M = 10$ for all test problems. irace was configured to run a maximum of 20,000 experiments for each experimental setup.

We implemented iRVEA and iRNSGA-II using a component-wise design in DESDEO. The list of tunable parameters considered in the experimentation, along with their corresponding ranges, is given in Table 1. These values are derived from the experimental setup presented in [41]. The crossover operator can be set as either SBX or blend crossover (BLX-α), with additional configurable parameters such as the crossover probability and, depending on the operator, the SBX distribution index or the α parameter. They both control the diversity of offspring with respect to the parent solutions. For SBX, a higher distribution index results in offspring that are close to the parents, while lower values promote diversity. For BLX-α, higher α values enable exploration, whereas lower values generate solutions closer to the parents. Mutation options include uniform and polynomial mutation, with adjustable parameters such as the mutation probability factor, polynomial mutation distribution index, and uniform mutation perturbation value. A higher distribution index in polynomial mutation results in smaller, more localized changes, whereas lower values allow for greater variation. In uniform mutation, a high perturbation value results in small adjustments, while lower values lead to larger deviations. The selection operator can be set to either random or tournament, with tournament size configurable between 2 and 10. Larger tournament sizes focus on exploitation by favoring better solutions, while smaller sizes encourage exploration by maintaining population diversity.

Reference points were generated using the ADM with iRVEA and iRNSGA-II, configured with their default evolutionary operators

and parameters as outlined in [25] and [20], respectively. We utilized four iterations for the learning phase and three iterations for the decision phase in all test instances.

Table 1: Parameters to be configured using irace.

Parameter name	Allowed values
Crossover	SBX, BLX-α
Crossover probability	[0.0, 1.0]
SBX crossover distribution index (δ)	[5.0, 400.0]
BLX-α crossover alpha value (α)	[0.0, 1.0]
Mutation	uniform, polynomial
Mutation probability factor	[0.0, 2.0]
Polynomial mutation distribution index (ρ)	[5.0, 400.0]
Uniform mutation perturbation (β)	[0.0, 1.0]
Selection	random, tournament
Tournament size (η)	[2, 10]

It is common practice among researchers to apply the same configuration from a posteriori methods to their interactive counterparts. In this context, we decided to compare the phase-specific configurations of iRVEA and iRNSGA-II with the optimal configurations for RVEA and NSGA-II, respectively. Both a posteriori methods were implemented using a component-wise design in DESDEO. The irace tool was then employed, using hypervolume as the performance indicator, to identify the optimal configurations, which we will refer to as "general configurations."

First, we examined the differences in the operators and parameters derived from the phase-specific and general configurations. Next, we evaluated iRVEA and iRNSGA-II using both configurations across all test problems. For the phase-specific configuration, each method employed four iterations for the learning phase and three iterations for the decision phase, applying the configuration optimized for each respective phase. For the general configuration, each method utilized seven iterations (four for the learning phase and three for the decision phase), using the same configuration throughout the entire solution process. The same reference points were used in both experiments, with the first four points more widely distributed across the Pareto front and the last three points focusing on a region of interest. Each method was run 20 times for each test problem and the median and standard deviation of the PHI_{RS} and PHI_{FD} were computed. To statistically compare the performance of the phase-specific configuration against the general configuration, we conducted a Wilcoxon rank-sum test [45] at a 5% significance level.

In the following sections, we present the results for each interactive method, highlighting the differences in the optimal configuration across phases and analyzing the performance of the phase-specific configuration in comparison to the general configuration.

4.1 Results for iRVEA

Table 2 summarizes the results for the phase-specific configuration of iRVEA and the general configuration of RVEA. The first column lists the parameters, while the second column indicates the number of objective functions (k). The third column shows the parameter values for the general configuration, and the fourth and fifth columns outline the phase-specific configurations.

The learning and decision phases employed distinct selection operators depending on the number of objective functions. For three objective functions, the learning phase used tournament selection, while the decision phase adopted random selection. For five objective functions, both phases used tournament selection, with a slightly larger tournament size in the decision phase (6 versus 7). For seven objective functions, the learning phase changed to random selection, while the decision phase used tournament selection. In contrast, the general configuration maintained random selection for the different numbers of objective functions, displaying less variation than the phase-specific configuration.

In the crossover operator, both configurations used BLX-α for different numbers of objective functions but with varying α values. For three and five objective functions, the decision phase used higher α values (0.97 and 0.98, respectively) than the learning phase (0.81 and 0.89, respectively). For seven objective functions, the α values in both phases were similar (0.983 and 0.986, respectively). The general configuration adopted slightly lower α values for three and five objective functions (0.76 and 0.81, respectively) but used a slightly higher α value for seven objective functions (0.9996).

The crossover probability showed marked differences between both phases. For three objective functions, it decreased from the learning phase (0.76) to the decision phase (0.30). For five objective functions, both phases maintained high probabilities (0.9979 in the learning phase and 0.83 in the decision phase). Similarly, for seven objective functions, the learning phase maintained a high probability (0.998), while the decision phase was slightly lower (0.84). The general configuration typically balanced these extremes, often falling between the values of the learning and decision phases. An exception occurs in the three-objective case, where the general configuration showed a higher crossover probability than both phases.

In terms of mutation, the learning and decision phases showed different behavior. For three objective functions, the learning phase used uniform mutation, while the decision phase changed to polynomial. For five objective functions, both phases used uniform mutation, but the decision phase employed a slightly reduced perturbation parameter (0.93 compared to 0.97 in the learning phase). For seven objective functions, the learning phase used polynomial mutation, while the decision phase retained uniform mutation with a higher perturbation value (0.989). The general configuration consistently used uniform mutation for the different numbers of objective functions, contrasting with the phase-specific configuration.

The mutation probability also varied for the different numbers of objective functions. For three objective functions, it decreased from the learning phase (0.18) to the decision phase (0.08). For five objective functions, it slightly increased in the decision phase (0.13 compared to 0.09 in the learning phase). For seven objective functions, the probability decreased slightly in the decision phase (0.16 compared to 0.18 in the learning phase). The general configuration generally exhibited higher mutation probabilities than either phase.

Overall, the learning phase prioritized exploration with higher crossover probabilities and the occasional use of polynomial mutation with a low perturbation value. In contrast, the decision phase focused on exploitation. Despite utilizing higher α values to maintain some level of exploration, the decision phase relied on lower

crossover probabilities and consistent use of uniform mutation with high perturbation values, promoting localized adjustments. The general configuration provided a balanced approach with more stable parameter settings.

Table 2: Results for the general configuration of RVEA and the phase-specific configuration of iRVEA

Parameter	k	General	Learning phase	Decision phase
Selection	3	Random	Tournament (η=6)	Random
	5	Random	Tournament (η=6)	Tournament (η=7)
	7	Random	Random	Tournament (η=3)
Crossover	3	BLX-α (α=0.7609)	BLX-α (α=0.8114)	BLX-α (α=0.9726)
	5	BLX-α (α=0.8163)	BLX-α (α=0.8889)	BLX-α (α=0.9844)
	7	BLX-α (α=0.9996)	BLX-α (α=0.9835)	BLX-α (α=0.9865)
Crossover probability	3	0.8969	0.7635	0.3023
	5	0.876	0.9979	0.8302
	7	0.7978	0.9984	0.8389
Mutation	3	Uniform (β=0.8609)	Uniform (β=0.659)	Polynomial (ρ=67.4145)
	5	Uniform (β=0.999)	Uniform (β=0.9672)	Uniform (β=0.9307)
	7	Uniform (β=0.9767)	Polynomial (ρ=6.4311)	Uniform (β=0.9887)
Mutation probability	3	0.8875	0.1801	0.0802
	5	0.9899	0.0886	0.1275
	7	0.9552	0.1843	0.1579

Figure 2 presents a summary of performance comparisons between configurations based on the median PHI_{RS} and PHI_{FD} values achieved by iRVEA. Blue cells represent cases where the phase-specific configuration performed better, while red cells indicate better performance by the general configuration. Statistically significant differences are marked with check marks, whereas crosses denote non-significant differences. Additional details, including the median and standard deviation of PHI_{RS} and PHI_{FD} values for each test problem, are provided in the supplementary material.

In the learning phase, the phase-specific configuration generally outperformed the general configuration. For three objective functions, significant differences were observed in all test instances except WFG6 and WFG9. For five objective functions, the phase-specific configuration performed better in most test problems, though the general configuration achieved significantly better results in WFG5 and WFG9. For seven objective functions, the phase-specific configuration outperformed in most test problems, except in DTLZ1, DTLZ3, DTLZ7, and WFG6, where no significant differences were observed.

In the decision phase, the phase-specific configuration also performed better across most test problems. For three objective functions, it consistently outperformed except in WFG6 and WFG9. For five objective functions, the phase-specific configuration was the best, except for WFG5, where the general configuration showed significantly better results, and WFG6 and WFG9, where no significant differences were observed. For seven objective functions, the phase-specific configuration outperformed in most cases, with no significant differences in DTLZ5, WFG3, WFG5 and WFG6.

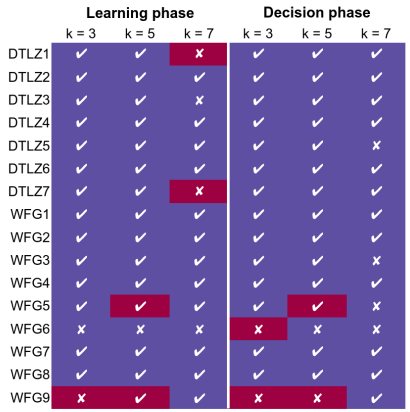


Figure 2: Results of the performance comparison between phase-specific and general configurations of iRVEA

4.2 Results for iRNSGA-II

Table 3 shows the results for the phase-specific configuration of iRNSGA-II and the general configuration of NSGA-II.

In the selection operator, the learning and decision phases showed varying behaviors. For three and seven objective functions, both phases used random selection. For five objective functions, the learning phase employed tournament selection with a size of 3, while the decision phase used a slightly larger size (4). The general configuration predominantly employed random selection but used tournament selection with a size of 3 for seven objective functions.

The learning and decision phases adopted different crossover operators. For three objective functions, the learning phase used BLX-α crossover with a high α value (0.92), while the decision phase employed SBX crossover with a high distribution index (126.65). For five objective functions, both phases utilized BLX-α crossover, but the decision phase adopted a higher α value (0.97 compared to 0.82 in the learning phase). For seven objective functions, the learning phase used BLX-α crossover, while the decision phase transitioned to SBX crossover with a high distribution index (280.60). The general configuration used SBX crossover for five and seven objective functions and BLX-α crossover for three objective functions.

The crossover probability highlighted further differences between phases. For three objective functions, the decision phase exhibited a higher probability (0.50) than the learning phase (0.39). For five and seven objective functions, the learning phase had higher crossover probabilities (0.982 and 0.9775, respectively) than the decision phase (0.33 and 0.71, respectively). The general configuration balanced these extremes in most of the cases.

Uniform mutation was used in both phases, with variation in the perturbation parameter. For three objective functions, the decision phase had a higher perturbation value (0.98) than the learning phase (0.91). For five objective functions, the decision phase showed a lower perturbation (0.03 compared to 0.07). For seven objective functions, the decision phase adopted a higher perturbation (0.92) than the learning phase (0.64). The general configuration used polynomial mutation for five objective functions while maintaining uniform mutation for three and seven objective functions.

The mutation probability varied similarly. For three objective functions, it decreased from the learning phase (0.29) to the decision phase (0.10). For five and seven objective functions, mutation probabilities increased in the decision phase (0.77 and 0.37, respectively) compared to the learning phase (0.17 and 0.14, respectively). The general configuration typically showed higher mutation probabilities, except in the case of seven objective functions.

Overall, the learning phase prioritized exploration utilizing smaller tournament sizes, higher crossover probabilities, and smaller uniform mutation perturbation. The decision phase emphasized exploitation, incorporating larger tournament sizes. While the distribution index and α values remained high to maintain some exploratory capability, the decision phase typically relied on lower crossover probabilities. The general configuration struck a balance between the two, offering more consistent parameter values for the different numbers of objective functions.

Table 3: Results for the general configuration of NSGA-II and the phase-specific configuration of iRNSGA-II

Parameter	k	general	Learning phase	Decision phase
Selection	3	Random	Random	Random
	5	Random	Tournament (η=3)	Tournament (η=4)
	7	Tournament (η=3)	Random	Random
Crossover	3	BLX-α (α=0.997)	BLX-α (α=0.9206)	SBX (δ=126.6523)
	5	SBX (δ=344.8321)	BLX-α (α=0.8169)	BLX-α (α=0.9682)
	7	SBX (δ=150.7357)	BLX-α (α=0.6928)	SBX (δ=280.5997)
Crossover probability	3	0.6847	0.3881	0.5029
	5	0.3445	0.9828	0.3258
	7	0.8829	0.9775	0.7053
Mutation	3	Uniform (β=0.932)	Uniform (β=0.9109)	Uniform (β=0.9797)
	5	Polynomial (ρ=124.1775)	Uniform (β=0.0747)	Uniform (β=0.0347)
	7	Uniform (β=0.4643)	Uniform (β=0.6431)	Uniform (β=0.9247)
Mutation probability	3	0.9695	0.2896	0.1015
	5	0.9493	0.1718	0.7668
	7	0.0637	0.1432	0.3695

Figure 3 summarizes the performance comparison results of iRNSGA-II using both configurations. Blue cells represent cases where the phase-specific configuration performed better, while red cells indicate instances where the general configuration outperformed. A check mark within a cell denotes a statistically significant difference, whereas a cross indicates no significant difference. Detailed median and standard deviation of the PHI_{RS} and PHI_{FD} values are included in the supplementary material.

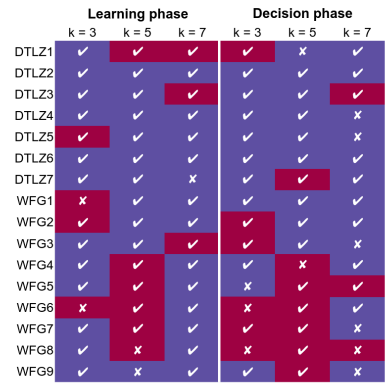


Figure 3: Results of the performance comparison between phase-specific and general configurations of iNSGA-II

In contrast to iRVEA, the results for iNSGA-II exhibit greater variability across both phases. In the learning phase, the phase-specific configuration outperformed the general configuration in most test problems. For problems with three objective functions, the phase-specific configuration performed better in most cases, with the exceptions of DTLZ5 and WFG2, where the general configuration was significantly better. No significant differences were found for DTLZ5, WFG1, WFG2 and WFG6. For five objective functions, the phase-specific configuration showed better performance in DTLZ2-7 and WFG1-3, while the general configuration was best for DTLZ1 and WFG4-7. No significant differences were found in WFG8 and WFG9. For seven objective functions, the phase-specific configuration performed better in all test instances except DTLZ1, DTLZ3 and WFG3, where the general configuration was significantly better. No significant difference was observed for DTLZ7.

In the decision phase, the phase-specific configuration outperformed the general configuration in most test problems. For three objective functions, the phase-specific configuration was significantly better in DTLZ2-7, WFG1, WFG4 and WFG9. However, the general configuration performed better in DTLZ1, WFG2, WFG3 and WFG7. There were no significant differences in WFG5, WFG6 and WFG8. For five objective functions, the phase-specific configuration performed better in DTLZ2-6 and WFG1-3, while the general configuration was better in DTLZ7 and WFG5-9. DTLZ1 and WFG4 did not show significant differences. For seven objective functions, the phase-specific configuration performed significantly better in most test problems except DTLZ3 and WFG5 where the general configuration showed better performance. In DTLZ4, DTLZ5, WFG3, WFG7, WFG8 and WFG9 there were no significant differences.

Overall, the phase-specific configuration performed better in both the learning and decision phases, particularly for test instances with three and seven objective functions. However, the general configuration was competitive and often best in WFG benchmark problems, especially for five objective functions, where it consistently performed well. These results highlight the complementary strengths of the phase-specific and general configurations across different test problems.

5 Discussion

The results of this study reveal notable differences between phase-specific and general configurations in iRVEA and iRNSGA-II. While both configurations showed strengths and weaknesses across various problems and different numbers of objective functions, the underlying trends provide insight into how these configurations balance exploration and exploitation.

For iRVEA, the phase-specific configuration showed a better performance in most test problems, particularly in the learning phase, where its parameters were tailored for more exploratory behavior. In contrast, the general configuration, which maintained consistent parameter settings across phases and the different numbers of objective functions, performed better in a few specific instances. These instances predominantly occurred in problems with higher complexity, such as those in the WFG benchmark, where maintaining consistent parameter settings may provide stability and prevent overfitting to specific regions of the Pareto front.

Similarly, for iRNSGA-II, the phase-specific configuration typically outperformed the general configuration, particularly for problems with three and seven objective functions. However, the general configuration of iRNSGA-II showed competitive performance in multiple problems from the WFG benchmark with five objective functions. Nevertheless, since iRNSGA-II was not inherently designed as an interactive method, the strong performance of its general configuration does not imply that the phase-specific configuration was ineffective. Instead, it highlights the need for further refinement of iRNSGA-II to ensure it can better adapt to the changing preferences of a decision maker. This is further supported by the median PHI_{RS} and PHI_{FD} values obtained by iRNSGA-II in WFG test problems, which suggest that the method may require improvements to enhance convergence to the Pareto front and adaptability to preference information.

The Wilcoxon test results for both methods support these observations. For iRVEA, the phase-specific configuration achieved statistically significant superiority in most cases, while the general configuration occasionally showed significant advantages in WFG problems. In contrast, the results for iRNSGA-II exhibited more variability. While the phase-specific configuration consistently performed well in both learning and decision phases, the general configuration demonstrated significant advantages in specific problems.

When analyzing the median PHI_{RS} and PHI_{FD} across all test problems, iRVEA consistently benefited more from phase-specific configurations than iRNSGA-II. In the learning phase, iRVEA, regardless of whether the phase-specific or general configuration was used, generally outperformed iRNSGA-II, especially in problems with more than three objective functions. In the decision phase, iRVEA again showed better performance with both configurations. However, there was a notable exception in problems with seven objective functions, where iRNSGA-II using the general configuration outperformed iRVEA with its corresponding general configuration. This highlights that the selection of a proper configuration is influenced not only by problem characteristics but also by the interactive method itself, suggesting the need for further investigation into optimal configuration tailored to different interactive methods.

6 Conclusions

We have proposed a framework for finding optimal configurations of interactive evolutionary multiobjective optimization methods considering the specific requirements of the learning and decision phases. By considering two interactive methods, iRVEA and iRNSGA-II, across a diverse set of benchmark problems with varying numbers of objective functions, we identified clear differences in the optimal configurations for each phase. Our results consistently showed that the learning phase benefits from configurations that prioritize exploration and diversity, while the decision phase requires configurations emphasizing exploitation to refine solutions. These findings suggest that phase-specific configurations can enhance the performance of interactive methods. This was a proof-of-concept study pointing to the need to investigate the topic further since utilizing the same configuration throughout the interactive solution process may lead to impaired performance.

While the findings are promising, there are limitations to consider in the future. The experiments were conducted on a finite set of benchmark problems that, while widely recognized in the field, may not comprehensively represent the characteristics of real-world problems. Future research should address these limitations by extending the scope of experiments to include more diverse test problems, particularly those that better reflect the characteristics of real-world problems. Investigating the proposed framework across interactive methods that employ varied types of preference information, as well as on problems with differing Pareto front shapes, constraints, and objective function scales, will further validate the generality of these findings (but this necessitates corresponding performance indicators and ADMs to be developed first). Such experiments will offer deeper insights into the configuration requirements of interactive methods, helping to improve their performance in the learning and decision phases. Additionally, future work could explore hybrid strategies that leverage the adaptability of phase-specific configurations while retaining the stability of general configurations, combining the best characteristics of both.

7 Acknowledgments

This research was supported by the Research Council of Finland (grant number 355346). The research is related to the thematic research area DEMO (Decision Analytics utilizing Causal Models and Multiobjective Optimization, jyu.fi/demo) of the University of Jyväskylä.

References

[1] B. Adenso-Díaz and M. Laguna. 2006. Fine-Tuning of Algorithms Using Fractional Experimental Designs and Local Search. *Operations Research* 54, 1 (2006), 99–114.

[2] B. Afsar, K. Miettinen, and A. B. Ruiz. 2021. An Artificial Decision Maker for Comparing Reference Point Based Interactive Evolutionary Multiobjective Optimization Methods. In *Evolutionary Multi-Criterion Optimization. EMO 2021,*

H. Ishibuchi, Q. Zhang, R. Cheng, K. Li, H. Li, H. Wang, and A. Zhou (Eds.). Springer, Cham, 619–631.

[3] B. Afsar, K. Miettinen, and F. Ruiz. 2021. Assessing the Performance of Interactive Multiobjective Optimization Methods: A Survey. *Comput. Surveys* 54, 4, Article 85 (2021).

[4] P. Aghaei Pour, S. Bandaru, B. Afsar, M. Emmerich, and K. Miettinen. 2024. A Performance Indicator for Interactive Evolutionary Multiobjective Optimization Methods. *IEEE Transactions on Evolutionary Computation* 28, 3 (2024), 778–787.

[5] P. Aghaei Pour, S. Bandaru, B. Afsar, and K. Miettinen. 2022. Desirable properties of performance indicators for assessing interactive evolutionary multiobjective optimization methods. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion. GECCO 2022*. ACM, New York, 1803–1811.

[6] P. Balaprakash, M. Birattari, and T. Stützle. 2007. Improvement Strategies for the F-Race Algorithm: Sampling Design and Iterative Refinement. In *Hybrid Metaheuristics. HM 2007*, T. Bartz-Beielstein, M. J. Blesa Aguilera, C. Blum, B. Naujoks, A. Roli, G. Rudolph, and M. Sampels (Eds.). Springer, Berlin, Heidelberg, 108–122.

[7] S. Bandaru and H. Smedberg. 2019. A parameterless performance metric for reference-point based multi-objective evolutionary algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference. GECCO 2019*. ACM, New York, 499–506.

[8] S. Bechikh, M. Kessentini, L. B. Said, and K. Ghédira. 2015. Chapter Four - Preference Incorporation in Evolutionary Multiobjective Optimization: A Survey of the State-of-the-Art. In *Advances in Computers*, A. R. Hurson (Ed.). Elsevier, 141–207.

[9] L. C. T. Bezerra, M. López-Ibáñez, and T. Stützle. 2012. Automatic Generation of Multi-objective ACO Algorithms for the Bi-objective Knapsack. In *Swarm Intelligence. ANTS 2012*, M. Dorigo, M. Birattari, C. Blum, A. Christensen, A. P. Engelbrecht, R. Groß, and T. Stützle (Eds.). Springer, Berlin, Heidelberg, 37–48.

[10] L. C. T. Bezerra, M. López-Ibáñez, and T. Stützle. 2015. Comparing Decomposition-Based and Automatically Component-Wise Designed Multi-Objective Evolutionary Algorithms. In *Evolutionary Multi-Criterion Optimization. EMO 2015*, A. Gaspar-Cunha, C. Henggeler Antunes, and C. Coello Coello (Eds.). Springer, Cham, 396–410.

[11] L. C. T. Bezerra, M. López-Ibáñez, and T. Stützle. 2016. Automatic Component-Wise Design of Multiobjective Evolutionary Algorithms. *IEEE Transactions on Evolutionary Computation* 20, 3 (2016), 403–417.

[12] M. Birattari, T. Stützle, L. Paquete, and K. Varrentrapp. 2002. A Racing Algorithm for Configuring Metaheuristics. In *Proceedings of the 4th Annual Conference on Genetic and Evolutionary . GECCO 2002*. Morgan Kaufmann Publishers, San Francisco, 11–18.

[13] M. Birattari, Z. Yuan, P. Balaprakash, and T. Stützle. 2010. F-Race and Iterated F-Race: An Overview. In *Experimental Methods for the Analysis of Optimization Algorithms*, T. Bartz-Beielstein, M. Chiarandini, L. Paquete, and M. Preuss (Eds.). Springer, Berlin, Heidelberg, 311–336.

[14] P.A.N. Bosman and D. Thierens. 2003. The Balance between Proximity and Diversity in Multiobjective Evolutionary Algorithms. *IEEE Transactions on Evolutionary Computation* 7, 2 (2003), 174–188.

[15] R. Cheng, Y. Jin, M. Olhofer, and B. Sendhoff. 2016. A Reference Vector Guided Evolutionary Algorithm for Many-Objective Optimization. *IEEE Transactions on Evolutionary Computation* 20, 5 (2016), 773–791.

[16] C. A. Coello Coello, G. B. Lamont, and D. A. Van Veldhuizen. 2007. *Evolutionary Algorithms for Solving Multi-Objective Problems* (2nd ed.). Springer, New York.

[17] S. P. Coy, B. L. Golden, G. C. Runger, and E. A. Wasil. 2001. Using Experimental Design to Find Effective Parameter Settings for Heuristics. *Journal of Heuristics* 7, 1 (2001), 77–97.

[18] K. Deb. 2001. *Multi-Objective Optimization using Evolutionary Algorithms*. John Wiley & Sons, Chichester.

[19] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197.

[20] K. Deb and J. Sundar. 2006. Reference Point Based Multi-Objective Optimization Using Evolutionary Algorithms. In *Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation. GECCO 2006*. ACM, New York, 635–642.

[21] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler. 2005. Scalable Test Problems for Evolutionary Multiobjective Optimization. In *Evolutionary Multiobjective Optimization: Theoretical Advances and Applications*, A. Abraham, L. Jain, and R. Goldberg (Eds.). Springer, London, 105–145.

[22] D. Doblas, A. J. Nebro, M. López-Ibáñez, J. García-Nieto, and C. A. Coello Coello. 2022. Automatic Design of Multi-objective Particle Swarm Optimizers. In *Swarm Intelligence. ANTS 2022*, M. Dorigo, H. Hamann, M. López-Ibáñez, J. García-Nieto, A. Engelbrecht, C. Pinciroli, V. Strobel, and C. Camacho-Villalón (Eds.). Springer, Cham, 28–40.

[23] A.E. Eiben, R. Hinterding, and Z. Michalewicz. 1999. Parameter Control in Evolutionary Algorithms. *IEEE Transactions on Evolutionary Computation* 3, 2 (1999), 124–141.

[24] M. Gong, F. Liu, W. Zhang, L. Jiao, and Q. Zhang. 2011. Interactive MOEA/D for Multi-objective Decision Making. In *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation. GECCO 2011*. ACM, New York, 721–728.

[25] J. Hakanen, T. Chugh, K. Sindhya, Y. Jin, and K. Miettinen. 2017. Connections of Reference Vectors and Different Types of Preference Information in Interactive Multiobjective Evolutionary Algorithms. In *2016 IEEE Symposium Series on Computational Intelligence. SSCI 2016*. IEEE, 1–8.

[26] Z. Hou, S. Yang, J. Zou, J. Zheng, G. Yu, and G. Ruan. 2018. A Performance Indicator for Reference-Point-Based Multiobjective Evolutionary Optimization. In *2018 IEEE Symposium Series on Computational Intelligence (SSCI), Proceedings*. IEEE, 1571–1578.

[27] S. Huband, P. Hingston, L. Barone, and L. While. 2006. A Review of Multiobjective Test Problems and a Scalable Test Problem Toolkit. *IEEE Transactions on Evolutionary Computation* 10, 5 (2006), 477–506.

[28] F. Hutter, H. H. Hoos, and K. Leyton-Brown. 2011. Sequential Model-based Optimization for General Algorithm Configuration. In *Learning and Intelligent Optimization. LION 2011*, C. A. Coello Coello (Ed.). Springer, Berlin, Heidelberg, 507–523.

[29] G. Lárraga and K. Miettinen. 2022. A General Architecture for Generating Interactive Decomposition-Based MOEAs. In *Parallel Problem Solving from Nature – PPSN XVII, Part II*, G. Rudolph, A. V. Kononova, H. Aguirre, P. Kerschke, G. Ochoa, and T. Tušar (Eds.). Springer, Cham, 81–95.

[30] G. Lárraga and K. Miettinen. 2025. Survey of interactive evolutionary decomposition-based multiobjective optimization methods. *Evolutionary Computation* to appear (2025).

[31] Y. Lavinas, M. Ladeira, G. Ochoa, and C. Aranha. 2022. Component-Wise Analysis of Automatically Designed Multiobjective Algorithms on Constrained Problems. In *Proceedings of the Genetic and Evolutionary Computation Conference. GECCO 2022*. ACM, New York, 538–546.

[32] K. Li, K. Deb, and X. Yao. 2018. R-Metric: Evaluating the Performance of Preference-Based Evolutionary Multiobjective Optimization Using Reference Points. *IEEE Transactions on Evolutionary Computation* 22, 6 (2018), 821–835.

[33] L. Li, H. Chen, J. Li, N. Jing, and M. Emmerich. 2018. Preference-based Evolutionary Many-objective Optimization for Agile Satellite Mission Planning. *IEEE Access* 6 (2018), 40963–40978.

[34] M. López-Ibáñez and T. Stützle. 2010. Automatic Configuration of Multi-objective ACO Algorithms. In *Swarm Intelligence. ANTS 2010.*, M. Dorigo, M. Birattari, G. A. Di Caro, R. Doursat, A. P. Engelbrecht, D. Floreano, L. M. Gambardella, R. Groß, E. Şahin, H. Sayama, and T. Stützle (Eds.). Springer, Berlin, Heidelberg, 95–106.

[35] M. López-Ibáñez and T. Stützle. 2012. The Automatic Design of Multiobjective Ant Colony Optimization Algorithms. *IEEE Transactions on Evolutionary Computation* 16, 6 (2012), 861–875.

[36] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, and T. Stützle. 2016. The irace Package: Iterated Racing for Automatic Algorithm Configuration. *Operations Research Perspectives* 3 (2016), 43–58.

[37] K. Miettinen. 1999. *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, Boston.

[38] K. Miettinen, F. Ruiz, and A.P. Wierzbicki. 2008. Introduction to Multiobjective Optimization: Interactive Approaches. In *Multiobjective Optimization: Interactive and Evolutionary Approaches*, J. Branke, K. Deb, K. Miettinen, and R. Slowinski (Eds.). Springer, Berlin, Heidelberg, 27–57.

[39] G. Misitano, B. S. Saini, B. Afsar, B. Shavazipour, and K. Miettinen. 2021. DESDEO: The Modular and Open Source Framework for Interactive Multiobjective Optimization. *IEEE Access* 9 (2021), 148277–148295.

[40] A. Mohammadi, M. N. Omidvar, and X. Li. 2013. A New Performance Metric for User-preference based Multi-objective Evolutionary Algorithms. In *2013 IEEE Congress on Evolutionary Computation. CEC 2013*. IEEE, 2825–2832.

[41] A. J. Nebro, M. López-Ibáñez, C. Barba-González, and J. García-Nieto. 2019. Automatic Configuration of NSGA-II with JMetal and Irace. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion. GECCO 2019*. ACM, New York, 1374–1381.

[42] A. J. Nebro, M. López-Ibáñez, J. García-Nieto, and C. A. Coello Coello. 2023. On the Automatic Design of Multi-objective Particle Swarm Optimizers: Experimentation and Analysis. *Swarm Intelligence* 18 (2023), 105–139.

[43] M. C. Riff and E. Montero. 2013. A New Algorithm for Reducing Metaheuristic Design Effort. In *2013 IEEE Congress on Evolutionary Computation. CEC 2013*. IEEE, 3283–3290.

[44] A. P. Wierzbicki. 1980. The Use of Reference Objectives in Multiobjective Optimization. In *Multiple Criteria Decision Making Theory and Application*, G. Fandel and T. Gal (Eds.). Springer, Berlin, Heidelberg, 468–486.

[45] F. Wilcoxon. 1992. Individual Comparisons by Ranking Methods. In *Breakthroughs in Statistics: Methodology and Distribution*, S. Kotz and N. L. Johnson (Eds.). Springer, New York, 196–202.

[46] G. Yu, J. Zheng, and X. Li. 2015. An Improved Performance Metric for Multi-objective Evolutionary Algorithms with User Preferences. In *IEEE Congress on Evolutionary Computation.CEC 2015*. IEEE, 908–915.

[47] E. Zitzler and L. Thiele. 1999. Multiobjective Evolutionary Algorithms: a Comparative Case Study and the Strength Pareto Approach. *IEEE Transactions on Evolutionary Computation* 3, 4 (1999), 257–271.